EJERCICIOS SELECCIÓN E ITERATIVA

EJERCICIO 1

Dispones de las variables:

```
python
edad = 22
tiene_entrada = True
esta_en_lista = False
es_empleado = True
```

Una persona puede entrar si cumple estas reglas:

- Tiene al menos 18 años.
- Tiene entrada o está en la lista.
- Si es menor de 25 años, debe estar en la lista o ser empleado.
- Si es empleado, puede entrar sin entrada, pero nunca si es menor de edad.

Determina si puede entrar y muestra un motivo concreto cuando se deniegue el acceso.

Construye una única expresión booleana para la autorización y utiliza condicionales adicionales para explicar el rechazo.

EJERCICIO 2

Dada una cadena texto, clasifícala según estas reglas:

- Está vacía o contiene únicamente espacios.
- Tiene menos de 5 caracteres útiles.
- Contiene exclusivamente dígitos.
- Representa un número entero positivo, aunque tenga espacios al principio o al final.
- En cualquier otro caso, es texto general.

Debes mostrar una categoría principal y no varias simultáneamente.

Prueba con " ", "1234", " 42 ", "0", "-12", "12.5" y "Python 3".

No uses expresiones regulares ni try/except.

EJERCICIO 3

Analiza una cadena que contiene dígitos, por ejemplo:

texto = "824456677123"

Recorre los caracteres y calcula:

- La longitud de la mayor secuencia de dígitos iguales consecutivos.
- Qué dígito forma esa secuencia.
- Cuántas secuencias de longitud 2 o más existen.

Para el ejemplo, la mayor longitud es 2. Hay secuencias de 44, 66 y 77.

Trabaja carácter por carácter, utiliza un bucle y no conviertas la cadena en una lista de grupos.

EJERCICIO 4

Empiezas con:

saldo = 850.0
cuenta_bloqueada = False
limite_diario = 600.0
retirado_hoy = 0.0

Simula una serie de operaciones de retirada y depósito. Puedes introducir las operaciones de ejemplo con su tipo y cantidad.

Reglas:

- No se aceptan cantidades iguales o inferiores a cero.
- No se puede retirar más dinero del disponible.
- No se puede superar el límite diario de retirada.
- Si la cuenta está bloqueada, no se permiten operaciones.
- Los depósitos aumentan el saldo, pero no modifican el límite de retirada.
- Una operación rechazada no debe modificar ninguna variable financiera.

Al final, muestra el saldo, el total retirado, el número de operaciones rechazadas y si la cuenta continúa bloqueada.

Comprueba que cada operación se valida antes de modificar el estado.

EJERCICIO 5

Evalúa una contraseña almacenada en una variable de tipo str.

Para que sea válida debe cumplir todas estas condiciones:

- Tener entre 10 y 20 caracteres.
- Contener al menos una letra mayúscula.
- Contener al menos una letra minúscula.
- Contener al menos un dígito.
- No contener espacios.
- No ser igual a "Python1234".

El programa debe comprobar cada requisito, indicar todos los que incumple y determinar si la contraseña es válida.
No uses expresiones regulares ni any() o all(). Utiliza bucles y variables booleanas.
Evita detener la comprobación en cuanto encuentres el primer error.

EJERCICIO 6

Construye un programa que muestre repetidamente este menú:

1. Consultar saldo
2. Ingresar dinero
3. Retirar dinero
4. Consultar movimientos
0. Salir

Requisitos:

- El menú se repite hasta elegir 0.
- Las opciones no válidas muestran un error, pero no terminan el programa.
- Los ingresos y las retiradas deben ser cantidades positivas.
- No se permite retirar más del saldo disponible.
- Se registra el número de ingresos y retiradas aceptados.
- El saldo nunca puede ser negativo.
- Al salir, muestra un resumen de la sesión.

Utiliza while, if/elif/else, int, float, str y bool. No uses clases ni funciones.

EJERCICIO 7

Implementa una versión sencilla de compresión por repetición consecutiva (RLE, Run-Length Encoding).

Por ejemplo:

Entrada: aaabbccccdaa

Salida: a3b2c4d1a2

El programa debe recorrer la cadena original y generar otra en la que cada grupo consecutivo se represente mediante su carácter y su cantidad.

Requisitos:

- Gestionar correctamente una cadena vacía.
- Detectar cambios de carácter.
- Procesar el último grupo, aunque no exista un carácter posterior que indique el cambio.
- Mantener el orden original.
- Contar cuántos grupos se han generado.
- Indicar si la cadena comprimida es más corta que la original.

No uses replace(), groupby() ni librerías externas.

Prueba con cadenas de un único carácter y con todos los caracteres distintos.